



YT-DLP Studio

Technical Documentation

Architecture, internals and maintenance/extension guide

Document version — 2026-09-28

Contents

1. Overview and tech stack
2. Project structure
3. Execution flow (what happens when the app opens)
4. Module by module
5. How yt-dlp commands are built (downloader.py)
6. The cover-crop mechanism (--ppa and escaping)
7. Dependency management (yt-dlp + FFmpeg)
8. Download queue and threading
9. Settings persistence
10. Internationalization (i18n)
11. Practical guide: adding/changing common things
12. Build and packaging (making the .exe)
13. Developer troubleshooting

1. Overview and tech stack

YT-DLP Studio is a desktop app written in **Python 3**, using **CustomTkinter** (a UI library on top of the standard Tkinter) for the interface. It does not reimplement any download logic: all the heavy lifting is done by the official **yt-dlp** and **FFmpeg** binaries, called as external processes through subprocess. In essence, the app is a command-line builder with a GUI, plus a queue/dependency manager around it.

Piece	Technology	Role
User interface	CustomTkinter (Tkinter)	Windows, tabs, forms
Download engine	yt-dlp (external binary)	Extraction and actual download
Media processing	FFmpeg / ffprompt (external)	Conversion, trimming, cover, mux
Packaging	PyInstaller	Builds the standalone .exe
Persistence	JSON (settings.json)	User settings

2. Project structure

```

ytdlp_studio/
|-- build.bat           # builds the .exe (Windows, via PyInstaller)
|-- requirements.txt
|-- make_logo.py        # regenerates icon/logo
|-- assets/
|   |-- icon.ico / icon.png      # app icon
|   |-- logo.png / logo_dark.png # horizontal logo (README, light/dark)
|   |-- theme_red.json          # black/white/red CustomTkinter theme
|-- docs/                    # PDFs (PT + EN, user + technical)
|-- docs_src/                # ReportLab scripts that generate the PDFs
`-- src/
    |-- main.py              # entry point, main window
    |-- theme.py              # shared colours (red, gray, outline)
    |-- i18n.py               # PT/EN strings
    |-- settings_store.py     # settings.json (read/write)
    |-- dependencies.py       # downloads/checks yt-dlp and FFmpeg
    |-- downloader.py         # yt-dlp commands + queue (thread) + playlist check
    |-- tabs.py               # Audio and Video tabs
    |-- converter_tab.py      # Convert tab (ffmpeg, compressor)
    |-- queue_tab.py          # Queue & Log tab
    |-- settings_dialog.py    # Settings window
    |-- help_dialog.py        # Help window
    |-- widgets.py            # RangeSlider (trim slider)
    `-- selfcheck.py          # offline tests

```

3. Execution flow

- 1 **main.py** creates the App(ctk.CTk) class, which loads SettingsStore (reads settings.json) and Translator (sets the UI language).
- 2 The top bar (`_build_topbar`) and content (`_build_content`) are built. The four tabs (AudioTab, VideoTab, ConverterTab, QueueTab) are created at once and stacked in the same grid cell — switching tabs is just `.tkraise()` on the chosen one.
- 3 300 ms after the window opens, `check_dependencies(initial=True)` runs in a separate *thread*, checking/downloading yt-dlp and FFmpeg without freezing the UI.

- 4 A `_poll_queue_events` loop runs every 150 ms via `self.after(150, ...)`, reading a `queue.Queue` where background threads (dependencies, downloads, playlist check, converter log) drop events. This is the only place widgets are updated — never directly from a thread.
- 5 When the user clicks Download/Add to queue, a `downloader.Job` is created and handed to `QueueManager`, which processes the queue sequentially in its own thread.

4. Module by module

main.py — App

Main window class. It builds the top bar (tabs + gear + help + theme + language), switches between the four tabs, opens the Settings/Help dialogs, runs dependency checks in a thread, and owns the `_poll_queue_events` loop that turns queue events into UI updates. It also holds `build_output_template(job)`, which decides the final path/name of each downloaded file (see section 5).

i18n.py

A `STRINGS = {'pt': {...}, 'en': {...}}` dictionary with every UI string by key. Translator keeps the current language and exposes `.t(key, **kwargs)`, which looks up the string and applies `.format(**kwargs)` when there are parameters (e.g. `tr.t('dep_downloading', size='~80MB')`).

settings_store.py

`SettingsStore` keeps a dictionary with every preference (folders, network/filter/behavior options, last values chosen on each tab, window geometry). `DEFAULTS` defines the default values — on load, saved values are merged on top of the defaults, so adding a new setting never breaks an old `settings.json` (the new key simply uses its default until changed). The file lives at `data/settings.json` next to the executable (see section 7 for how that folder is resolved, with a fallback to `%APPDATA%`).

dependencies.py

Resolves the paths of `yt-dlp.exe/ffmpeg.exe/ffprobe.exe` inside a `bin/` folder next to `settings.json`. `check_ytdlp()/check_ffmpeg()` run `--version` to confirm the binary exists and works. `install_ytdlp()` downloads the latest executable straight from GitHub Releases; `install_ffmpeg()` downloads the 'essentials' package (Windows) and extracts just the executables from the `.zip`. `DependencyManager` orchestrates this and reports progress through a `status_cb(component, state, extra)` callback.

downloader.py

The most important module for maintenance. It contains the UI→yt-dlp flag mapping dictionaries (e.g. `RES_HEIGHT`, `VCODEC_FILTER`, `AUDIO_QUALITY_VALUE`), `build_command(job, output_template)` which builds the final argument list, `probe_media_info()` (title/artist/duration lookup), `check_playlist()` (availability check), the playlist cover helpers, and `QueueManager`, which runs jobs in a background thread. Detailed in section 5.

tabs.py

`DownloadTabBase` is the parent class with everything Audio and Video share: link box, trim slider, folder buttons, playlist-check button, Download/Add to queue, status and progress bar. `AudioTab` and `VideoTab` only implement `build_options_row()` (their specific fields) and `gather_tab_options()` (the dict that becomes

job.options).

converter_tab.py

The Convert tab. `build_ffmpeg_cmd()` builds the ffmpeg command from two tables, `probe_duration()` reads a file's duration with `ffprobe`, and `ConverterTab` runs ffmpeg in a thread with its own result queue. Detailed in section 6.

queue_tab.py

The Queue & Log UI: `DepStatusRow` (coloured dot + status text), `QueueRow` (one queue item with its own progress bar) and `QueueTab`, which lays it all out, including the log panel (`append_log`).

settings_dialog.py / help_dialog.py

Standalone CTKToplevel windows. `SettingsDialog` builds each section (Folders, Network, Filters, Behavior, Extra files) from small helper classes (`LabeledEntry`, `LabeledCheck`) that load/save their own value from/to `SettingsStore`. `HelpDialog` just renders the static `HELP_CONTENT` (a pt/en dictionary of sections).

widgets.py

`RangeSlider`: a hand-drawn two-handle slider on a `tkinter.Canvas` (`CustomTkinter` has no native component for this). `Helpers.seconds_to_hms/hms_to_seconds` convert between seconds and the HH:MM:SS shown in the text fields. Since the `Canvas` is plain `Tk` (not `CustomTkinter`), it doesn't follow runtime light/dark theme changes on its own — that's why `refresh_theme()` exists, called explicitly by `App.toggle_theme()` to recompute the canvas background.

theme.py + assets/theme_red.json

The black/white/red palette (`#FF0000`, matching yt-dlp's identity) has two layers: `assets/theme_red.json` is a full `CustomTkinter` theme (same schema as the library's built-in 'blue' theme, with the blues swapped for red), loaded once in `main.py` via `ctk.set_default_color_theme(...)` — covering every widget without an explicit colour. `theme.py` adds Python constants (`PRIMARY_KW`, `CANCEL_KW`, `SECONDARY_KW`) for the few buttons that need a different style — notably `CANCEL_KW`, a 'ghost' button (transparent fill, red outline) so cancel actions don't look like the solid red primary buttons. To change the red used across the app, edit these two sources; no theme colour is scattered elsewhere.

5. How yt-dlp commands are built

Each download is a `downloader.Job`: mode ('audio' or 'video'), URL, optional title/artist, and an options dict. That dict is built in `tabs.py` → `DownloadTabBase._make_jobs()` by merging **all** global settings (`dict(self.app.settings.data)`) with the tab-specific options (format, resolution, etc.) — so inside `build_command`, `s = job.options` reads both global settings and tab options alike.

`build_command()` always assembles in the same order:

- 1 Base flags: yt-dlp path, URL, `--newline --no-color --no-warnings --ignore-errors`, FFmpeg location, the two `--print` markers, and the output template (`-o`).
- 2 General behavior flags (`restrict-filenames`, `no-playlist`, `download-archive`, `geo-bypass`, etc.) — each checking `s.get('key')`.

- 3 Numeric network/robustness flags (parallel fragments, retries, speed limit, proxy, playlist-items, date/size/duration filters).
- 4 Trimming: if trim_start/trim_end were set by the slider, adds --download-sections '*START-END' --force-keyframes-at-cuts.
- 5 Extra files (metadata, chapters, description, thumbnail rules).
- 6 Audio block (-x --audio-format --audio-quality, metadata via --postprocessor-args) **or** video block (-f format selector, container, subtitles) — never both.
- 7 Finally, if the user set an advanced template in Settings → Network, it replaces the default -o.

Video format selector example

build_format_selector() turns Resolution/FPS/Codec into yt-dlp -f filters, always with a fallback so a download never fails from over-restriction:

```
# 1080p + H.264 becomes:
bestvideo[height<=1080][vcodec~='^avc1']+bestaudio
/best[height<=1080]
```

It first looks for separate video+audio streams matching the filters; if none exist, it falls back to the best single stream within the requested resolution.

Playlist folder and live status (--print)

Before each real job, QueueManager._resolve_playlist_folder() runs a quick separate command — the same strategy as the original .ps1 script — to find out whether the link is a playlist:

```
yt-dlp --get-filename --no-warnings -o "%(playlist_title)s"
URL --playlist-items 1
```

If the result isn't empty, "NA" or the URL itself, job.options['folder'] is replaced by a subfolder with that name (sanitized by sanitize_filename()) and job.title/job.artist are cleared (they were filled by the lookup with the first track; left in place, every track would get the same file name and -metadata title= and overwrite each other). This must run **before** output_template_fn(job) in _run_job().

For the live "Title - Artist (X of Y)" status, build_command() always adds two --print flags, using yt-dlp's before_dl (before each video starts) and after_move (after the final file is in place) hooks, with their own prefix (BDL@@@/ADL@@@) and @@@-separated fields that are easy to match with NOW_PLAYING_RE:

```
--print before_dl:BDL@@@%(title)s@@@%(artist|)s@@@%(playlist_index|)s@@@%(playlist_count|)s
--print after_move:ADL@@@%(title)s@@@%(artist|)s@@@%(playlist_index|)s@@@%(playlist_count|)s
```

%(artist|)s uses only the real artist metadata (mostly present on YouTube Music links), defaulting to an empty string — **deliberately without falling back to uploader (channel name)**. An earlier version used %(artist,uploader,creator|)s, which turned the names of channels that re-upload remixes/covers into a wrong prefix. When one of these lines arrives in _run_job()'s stdout loop, a now_playing event is emitted and handled in App._handle_queue_event(), which builds the localized text into job.current_label.

Automatic title/artist/duration lookup

When exactly one link is pasted, DownloadTabBase schedules downloader.probe_media_info(url) 900 ms after the last keystroke (_on_link_changed cancels and reschedules self.after() on every key — a simple

debounce), running in a separate thread. The command downloads nothing:

```
yt-dlp --print "%(title|)s@@@%(artist|)s@@@%(duration|0)s"
--skip-download --playlist-items 1 URL
```

The result returns through a per-tab queue. Queue read every 200 ms by `_poll_probe_results`. A sequence number (`self._probe_seq`) discards stale results if the link changes before the lookup ends.

`_apply_probe_result()` updates the duration/slider and, on the Audio tab, title/artist (only if still empty). With more than one link, `_start_probe()` does nothing, since title/artist/trim apply to every item of a batch.

Final file name: two separate rules

`App.build_output_template(job)` in `main.py` decides the `-o` template, and audio and video deliberately follow different rules:

```
if job.mode == "audio":
    # title ONLY, never the artist, even if job.artist exists
    return f"{folder}/{job.title or '%(title)s'}.{%(ext)s}"
# video: title + channel, resolved by yt-dlp itself
return f"{folder}/{%(title)s - %(channel,uploader)s}.{%(ext)s}"
```

YouTube's artist field isn't always reliable, so the audio file name **never** uses `job.artist`; the Artist field still goes to `--postprocessor-args` (ID3 metadata), so this naming policy doesn't affect the internal tags. For video, `%(channel,uploader)s` is resolved entirely by yt-dlp at download time.

Thumbnail rules: JPG and 1:1 crop

When to save/convert/crop a thumbnail is decided in a few lines at the start of `build_command()`. There's no separate setting: for audio the square crop is always automatic, for both the embedded and the saved cover (an earlier 'Square 1:1 cover' toggle was removed as redundant):

```
save_thumb_file = ... # 'Save thumbnail' OR (playlist + save cover)
want_thumbnail = save_thumb_file or embed_metadata_thumb
want_square_crop = job.mode == "audio" and want_thumbnail
want_jpg_convert = save_thumb_file or want_square_crop
```

The crop is always gated on `job.mode == "audio"` — a video thumbnail is never cropped.

`--embed-thumbnail` is controlled only by 'Embed metadata and cover art' (on by default).

Playlist cover and availability check

Playlist cover: with a playlist detected and 'Save thumbnail' (or 'Save cover in folder') on, `build_command()` adds `-o "pl_thumbnail:FOLDER/playlist_thumb.%(ext)s"` (a documented yt-dlp output template type) so the playlist's own cover gets a fixed name. When the job finishes, `promote_folder_cover()` prefers that file (falling back to the oldest image if the playlist has no cover), converts it to `playlistcover.jpg` — cropped 1:1 for audio — and `cleanup_stray_thumbnails()` deletes the per-track covers.

Availability check: `downloader.check_playlist(url, emit)` is a port of the PowerShell script: `--flat-playlist --print "%(id)s%(title)s"` maps position/title, then `-i -s --print OK:%(id)s` simulates the whole playlist, and every `ERROR: [x] ID: reason` line (regex `PL_ERR_RE`) becomes a Log line with the neighbouring tracks. It runs in a thread; `emit` is the same thread-safe Log queue.

6. The cover-crop mechanism (--ppa and escaping)

The automatic 1:1 crop of music covers uses yt-dlp's --ppa NAME:ARGS mechanism to inject extra FFmpeg arguments into the thumbnail conversion step (ThumbnailsConvertor postprocessor), applying a centered crop filter:

```
crop='if(gt(ih,iw),iw,ih)':'if(gt(iw,ih),ih,iw)'
```

The catch is that yt-dlp runs its own shlex.split() on the --ppa value before passing it to FFmpeg — and that split strips bare single quotes, which are exactly what protects the expression's commas from being read as filter separators by FFmpeg. The fix (SQUARE_CROP_PPA, downloader.py) wraps each literal single quote in double quotes (""), a classic POSIX shell quoting trick that survives shlex.split:

```
_Q = the 3-character string: " ' "
SQUARE_CROP_PPA = (
    "ThumbnailsConvertor+FFmpeg_o:-c:v mjpeg -vf "
    f"crop={_Q}if(gt(ih,iw),iw,ih){_Q}:{_Q}if(gt(iw,ih),ih,iw){_Q}"
)
```

This was validated by running shlex.split() on the final value and checking that the resulting token is exactly crop='if(gt(ih,iw),iw,ih)':'if(gt(iw,ih),ih,iw)'. **If this string ever changes, re-validate it with a shlex test before trying real yt-dlp** — it's easy to break the escaping without noticing. (The playlist cover crop in promote_folder_cover() calls ffmpeg directly with an argv list, so it needs no extra escaping.)

Convert tab (converter_tab.py)

build_ffmpeg_cmd() builds the command from two tables (AUDIO: bitrate per quality; VIDEO: CRF per quality) and caps resolution with scale=-2:'min(ih,H)' (never upscales). Progress comes from -progress pipe:1: duration from the Duration: line and progress from out_time_us=, both on stdout with stderr merged. A new format is one line in AUDIO/VIDEO.

Target-size compressor: with target_bytes, probe_duration() (ffprobe) reads the duration and the bitrate becomes bytes×8/duration×0.97 (3% headroom for the container). For video it subtracts the table's -b:a and uses -b:v/-maxrate/-bufsize in a single pass (the trailing -b:v overrides VP9's -b:v 0, since the last option wins in ffmpeg). Impossible targets (<100 kbps of video) or FLAC/WAV raise ValueError, shown in the Log. If exact size is ever needed, the next step is 2-pass encoding (-pass 1/2).

src/selfcheck.py tests the converter command, the compressor bitrate, the playlist cover template and the availability parser, all offline.

7. Dependency management

yt-dlp and FFmpeg are **not** bundled in the .exe — they're downloaded on first run into a data/bin/ folder right next to the executable (portable: easy to find, copy or delete — nothing hidden in AppData). This keeps the app small and makes sure users always have a reasonably recent yt-dlp (which needs frequent updates to keep up with site changes).

settings_store.get_app_data_dir() resolves that folder from sys.executable when running as a packaged .exe (never from __file__, which would point to PyInstaller's temp folder in --onefile mode), with an automatic fallback to %APPDATA%/YT-DLP Studio if the folder next to the .exe can't be created/written (e.g. installed in Program Files without admin rights).

Binary	Source	Approx. size
yt-dlp.exe	github.com/yt-dlp/yt-dlp/releases/latest	~15 MB
ffmpeg.exe / ffprobe.exe	gyan.dev/ffmpeg/builds (essentials)	~80 MB

The 'Update yt-dlp' button simply re-runs `install_ytdlp()`, overwriting the local binary with the latest release. If either domain ever changes, update the `YTDLP_RELEASE` / `FFMPEG_WIN_ESSENTIALS` constants at the top of `dependencies.py`.

8. Download queue and threading

The project's golden rule: **no background thread touches a widget directly**. All communication goes through a queue. Queue (`QueueManager.events`, plus per-tab queues for the lookup and the converter), and only the main thread (the one running Tkinter's `mainloop()`) reads them, via `self.after()` polling. This avoids Tkinter's classic freezes/crashes when a thread touches the UI.

`QueueManager.start()` launches a daemon thread (`_run_loop`) that walks `self.jobs` in order, calling `_run_job()` for each. Each job runs yt-dlp via `subprocess.Popen`, reads its output line by line, extracts the percentage with a regex (`PROGRESS_RE`) and emits `job_update/log` events for every new line.

'Cancel Current Download' sets a `threading.Event` (`_stop_flag`) and terminates the running yt-dlp process; this cancels **only the current item** — the flag is cleared right after, so the rest of the queue keeps going.

9. Settings persistence

A single JSON file holds everything: default folders, every advanced option, and the last value chosen in each tab's dropdowns (last_* keys), so the app reopens the way the user left it. `SettingsStore.save()` is called explicitly after any relevant change (there's no field-by-field autosave).

10. Internationalization (i18n)

Every visible string lives in `i18n.STRINGS['pt']` and `['en']`, with the same key in both. Each screen exposes a `retranslate()` method that resets the text of every widget stored in its label map (widget→key) — called whenever the language changes (PT/EN button).

Adding a new language

- 1 Copy the 'en': {...} block in `i18n.py`, translate every value and give it a new key (e.g. 'es'). Do the same for `HELP_CONTENT` in `help_dialog.py`.
- 2 In the language button (`main.py` → `toggle_language`), replace the binary 'pt'/'en' logic with a list of supported languages.

11. Practical guide: adding/changing common things

Add a new audio download format

- 1 In `tabs.py` → `AudioTab.AUDIO_FORMATS_TECH`, add the label shown in the menu (e.g. 'WMA').
- 2 In `downloader.py` → `AUDIO_FORMAT_VALUE`, map it to the value yt-dlp expects in `--audio-format` (e.g. 'WMA': 'wma').

Add a new video resolution

- 1 In `tabs.py` → `VideoTab.RESOLUTIONS_TECH`, add the label (e.g. '8640p (16K)').
- 2 In `downloader.py` → `RES_HEIGHT`, map it to the height in pixels (e.g. '8640p (16K)': 8640).

Add a new Convert format

One line in `converter_tab.py`: an AUDIO entry (extension, codec args, three bitrates or None for lossless) or a VIDEO entry (extension, codec args including `-b:a`, three CRF values).

Add a new Settings option

- 1 Add the default in `settings_store.py` → `DEFAULTS`.
- 2 Add the PT/EN label strings in `i18n.py`.
- 3 Add the field in `settings_dialog.py` (one entry in `self.entries[...]` or `self.checks[...]` in the right section — `_load_values/_save` already iterate those dicts).
- 4 Use the key inside `build_command()` in `downloader.py` (`s.get('your_key')`).

Change the in-app Help text

Edit `HELP_CONTENT['pt'] / ['en']` in `help_dialog.py` — each item is a (section title, text) pair.

Regenerate these PDFs

Run the scripts in `docs_src/` (`make_user_guide.py`, `make_tech_doc.py` and their `_en` versions). They only need ReportLab and write straight into `docs/`.

12. Build and packaging (making the .exe)

The project ships as Python source + `build.bat`, not a prebuilt `.exe`, because PyInstaller packages for the platform it runs on: building a Windows `.exe` means running the build on Windows.

What `build.bat` does, step by step:

- 1 Creates a Python virtual environment (`.venv`).
- 2 Installs `requirements.txt` (`customtkinter`, `pillow`, `pyinstaller`).
- 3 Runs PyInstaller in `--onefile --windowed` mode, bundling the `assets/` folder (icon, logo, theme) inside the executable.
- 4 The result is `dist/YT-DLP Studio.exe` — a single file, no Python needed on the target machine.

At runtime the app finds its own assets with `resource_path()` (`main.py`), which checks `sys._MEIPASS` — the temp folder PyInstaller creates for a `--onefile .exe`. Outside the `.exe` (running `python src/main.py`), it falls back to the `assets/` folder next to the project.

13. Developer troubleshooting

Symptom	Likely cause / where to look
App opens but freezes when clicking something	A background thread is calling a widget method directly — make sure new code goes through <code>queue.Queue + self.after()</code> instead of touching the UI.
A download always fails with the same error	Run the exact command from the Log panel in a terminal — the log prints the full '\$ yt-dlp ...' line before every run (the converter logs its '\$ ffmpeg ...' line the same way).
A new UI option has no effect on the download	Check that the key used in <code>tabs.py/settings_dialog.py</code> is exactly the one read in <code>downloader.py</code> → <code>build_command()</code> (key typos fail silently).
Something broke after a change	Run <code>python src/selfcheck.py</code> — it fails with an <code>AssertionError</code> pointing at the broken piece, no network needed.
PyInstaller builds the .exe but it won't open	Build once without <code>--windowed</code> to see the traceback in a console; usually it's a missing <code>--hidden-import</code> or an asset not included in <code>--add-data</code> .